Klaro — Study Guide

Page-by-page revision notes for the Major Project Part-1 report

Personal revision notes – Yasharth Singh

How to use this guide

This file walks through the submitted report (`major.pdf`) in the same order, chapter by chapter, page by page. Each chapter section below starts with the **report page number** so you can flip between this guide and the actual report. Read the Glossary first – every technical term used anywhere in the report is defined there. The last section explains, one by one, what every paper in the References list actually says.

Contents

1 Glossary – every term used in the report **2**
1.1 Abbreviations (report p. ix) . . . 2
1.2 Nomenclature (report p. x) . . . 2
1.3 Extra terms used in the report body (not in the official lists) . . . 3

2 Chapter 1: Introduction (report p. 1) **5**

3 Chapter 2: Background Study (report p. 2–3) **6**

4 Chapter 3: Requirement Analysis (report p. 4) **8**

5 Chapter 4: Detailed Design (report p. 5–7) **9**

6 Chapter 5: Implementation (report p. 8–10) **11**

7 Chapter 6: Experimental Results and Analysis (report p. 11–14) **13**

8 Chapter 7: Conclusion and Future Scope (report p. 15–16) **16**

9 References – what every paper actually says (report p. 17) **18**

10 Cheat Sheet – every number in one place **21**

1 Glossary – every term used in the report

1.1 Abbreviations (report p. ix)

4B 4 billion parameters – a way of stating how big an AI model is. More parameters generally means the model is more capable but also bigger to download and slower to run.

AI Artificial Intelligence.

CMMI Capability Maturity Model Integration – a certificate that proves a software company follows a mature, disciplined development process. RFPs sometimes require bidders to hold one.

GB Gigabyte – unit of file/download size.

LoRA Low-Rank Adaptation – a cheap way to further train ("fine-tune") a large AI model without retraining all of its weights. See Section 9 for the full idea.

PDF Portable Document Format – the file format RFPs come in.

RAG Retrieval-Augmented Generation – a technique where the AI first looks up the most relevant pieces of text and is only shown those, instead of the whole document. See Section 9.

RFP Request for Proposal – the tender document Klaro analyzes.

TF-IDF
Term Frequency–Inverse Document Frequency – a simple, classic way to score how important a word is to a document: a word scores high if it appears often in *this* document but rarely across other documents.

1.2 Nomenclature (report p. x)

Agent An AI helper with one job, e.g. "write the summary." Klaro's Phase 1 has one agent (the Summary agent); the full plan has three (Requirements, Executive Summary, Starter Pack – Chapter 7).

Part (chunk)
A piece of the RFP, about 1,800 words (roughly four pages), that the AI reads in one go. Long RFPs are cut into several parts because the AI model cannot read an 84-page document all at once.

Fact One short statement pulled out of the RFP, always kept together with the page number it came from.

Executive summary
The short, nine-heading overview of the RFP that Klaro produces – the main deliverable.

Pipeline
The fixed sequence of steps every RFP goes through, start to finish (Figure 4.3: extract → split → analyze → combine → write → check → show).

Local model
An AI model that runs on the user's own computer, never sending data over the internet – as opposed to a cloud AI service like a public chatbot.

Parameter
One of the internal numbers an AI model is made of (its learned weights). More parameters usually means a bigger, slower, but often more capable model.

Word piece (token)
The small unit of text an AI model actually reads and writes – often part of a word rather than a whole word. Model speed is usually measured in tokens ("word pieces") per second.

Backed by facts
Describes a summary point that matches a fact actually found in the RFP (as opposed to something invented).

Made-up information
A statement the AI produced that is *not* in the source document. Also called **hallucination** – see Section 9.

Precision
Of everything the AI listed as a requirement, the share that were actually correct. High precision = few wrong answers.

Recall Of everything that *should* have been listed, the share the AI actually found. High recall = few missed answers.

Benchmark
A fair test that gives several AI models the exact same task so their scores can be compared directly.

1.3 Extra terms used in the report body (not in the official lists)

F1-score A single number that balances precision and recall (their harmonic mean). Used in Table 6.1 because looking at precision or recall alone can be misleading – a model that lists everything gets perfect recall but terrible precision, and F1 punishes that.

Chunking / chunk overlap
Splitting a long document into parts (chunks). Klaro overlaps neighbouring parts by 200 words so a sentence that happens to fall right on the cut boundary is not lost or split awkwardly (report p. 6, Section 4.5).

Issuer detection
Klaro's way of working out *who published* the RFP: it looks for the organization name that appears most often, with a bias toward names ending in words like "Bank," "Authority," or "Ministry" (report p. 8, Section 5.4).

Grounding
A quality-check score (Figure 6.3) measuring what share of summary points are actually backed by an extracted fact.

Traceability
A quality-check score measuring what share of summary points carry a correct page number.

Coverage A quality-check score measuring what share of the collected facts made it into the final summary.

Duplication
A quality-check score measuring how much repeated content is in the summary.

TextRank A graph-based algorithm (see Section 9) for picking out the most important sentences or keywords in a text, used by Trappey et al. [2] and explained fully by Mihalcea & Tarau [3].

Transformer / self-attention
The neural-network architecture behind essentially every modern AI language model, including Qwen, Gemma and Phi (see Vaswani et al. [5], Section 9).

Fine-tuning

Continuing to train an already-trained AI model on new, task-specific examples so it gets better at that specific task. LoRA (below) is a cheap way to do this.

2 Chapter 1: Introduction (report p. 1)

Big picture

An RFP is long (50–100 pages) and its important details are scattered across it. You cannot safely upload a confidential RFP to a public AI chatbot. Klaro's answer: run a small AI model *entirely on the user's own laptop* and produce a nine-heading, page-referenced summary.

1.1 General

Defines what an RFP is and lists the things someone must find in it by hand today: issuer, work required, who can bid, deadlines, payment terms, contract length, special conditions. States the two real test RFPs used throughout the report are 54 and 84 pages long. Explains *why* cloud AI is out (confidentiality) and states Klaro's core idea: the AI model runs locally, so the document "never leaves it" (Figure 1.1).

Figure 1.1 ("Klaro in one picture"): shows the RFP PDF going into the Klaro desktop app, which talks to a small AI model, both boxed inside "your laptop." A separate box for "Online AI services" sits outside with a dashed arrow crossed out and labelled "never uploaded" – the single clearest visual statement of Klaro's privacy design.

1.2 Problem Statement

One formal sentence version of the whole project: given an RFP PDF, produce a short, nine-heading, page-referenced summary; must run on an ordinary laptop using only a local AI model; must never send any part of the document over the internet; must fail loudly (not silently) when something goes wrong.

1.3 Objectives (memorize these four)

1. Build the step-by-step pipeline (PDF → page-referenced summary).
2. Pick the best small AI model by testing several on the same task.
3. Measure summary quality and report weaknesses honestly.
4. Package it as a desktop app for Windows and macOS with progress updates and a side-by-side original-document view.

1.4 Scope of the Present Work

This phase = English, text-based PDFs only, one feature (the executive summary). German is the *first planned* additional language (mid-term goal, not done yet). Scanned files, Word files, requirement checklists, proposal drafts and searching past proposals are all future work (Chapter 7).

Key numbers to remember

54 and 84 pages (the two real test RFPs) · nine fixed summary headings · 4 objectives · scope = English + text PDFs only, this phase.

3 Chapter 2: Background Study (report p. 2–3)

Big picture

Five research papers were studied that are "close to our problem." None of them do what Klaro does: summarize a single confidential tender, on a laptop, with every point traceable to a page. Table 2.1 on p. 3 is the one-line summary of this whole chapter – learn that table.

2.1 General

Two eras of document analysis: older methods counted words and picked important sentences (extractive, statistical); newer methods use AI language models that can read and understand long text (abstractive, generative). Klaro is in the second camp, but borrows ideas from the first.

2.2 Earlier Work on Tender Documents

Beason et al. [1]
Processed many US government RFPs to spot technology trends. Found: RFPs have no consistent layout and are full of noise (contents pages, boilerplate, legal text) that must be cleaned out first; and structured, exact facts are more useful to readers than a free-flowing paragraph summary. **Klaro takes both lessons**: it skips contents pages and produces fixed headings instead of free prose.

Trappey et al. [2]
Summarized over 1,300 power-transformer tenders using TF-IDF (keyword scoring) + TextRank (sentence ranking). Keeping the top 35% of sentences preserved about 80% of the important requirements and cut reading time to about 29% of the original. **Klaro borrows**: summarize topic-by-topic rather than the whole document at once, and aim for a genuine reduction in length (this is exactly the weakness Klaro found in its own long-RFP results – Section 6.4.2).

Knisely & Pavliscsak [4]
Sorted research proposals into themes using *semi-supervised* clustering: a small number of expert-labelled examples guided the computer while it sorted the rest, which worked much better than sorting with no human input at all. **Klaro connection**: Klaro's own results showed the AI sometimes files a fact under the wrong heading, which is why Phase 2's plan (Section 7.3.3) starts with building a small human-checked ("golden") reference set.

2.4 Reading Very Long Documents

Standard AI models can only look at a limited amount of text at once, and the computational cost grows quickly as the text gets longer (this is the quadratic cost of self-attention – see Vaswani et al. [5] below). Longformer [6] fixes this so a model can read thousands of words in one go. Douzon et al. [7] go further for business documents by also using page layout, so a value in a table stays linked to its heading – but this is too heavy to run on a laptop. **Klaro's own answer**: since a small laptop model cannot read an 84-page RFP in one go anyway, Klaro deliberately cuts the document into parts, but keeps a page number on every single piece so nothing loses its source.

2.5 Small AI Models and Made-Up Answers

Language models sometimes state things that are not in the text they were given [8, 9], and they pay less attention to the middle of very long inputs [10] (see "Lost in the Middle," Section 9). One standard remedy is retrieval-augmented generation (RAG) [11] – look up only the relevant passages first, then show the model only those. **Klaro's remedy**: force the model to cite a page for every

point, then automatically check each point against the collected facts afterward. A model can later be specialized for a task cheaply via LoRA [12] (this is exactly the Phase-2 plan).

2.6 Summary of Gaps – Table 2.1 (memorize this table)

Work	**What it did**	**What it did not cover / was missing**
Beason et al. [1]	Found technology trends across many RFPs	No summary of a single RFP
Trappey et al. [2]	Picked key sentences from equipment tenders	Only copies sentences; one industry
Knisely & Pavliscsak [4]	Sorted proposals with a little expert help	Research proposals, not tenders
Longformer [6]	Let AI read very long text	Needs powerful hardware
Douzon et al. [7]	Used page layout for business documents	Too heavy for a laptop

One-line takeaway: none of these works on confidential tenders on an ordinary laptop, and none produces a summary where every point is linked to its page – that gap is exactly what Klaro fills.

4 Chapter 3: Requirement Analysis (report p. 4)

3.1 Who Uses Klaro

Primary user: a bid/proposal analyst, briefing a manager, with no programming background. The team also runs the same pipeline from the command line to test changes.

3.2 What the System Must Do – Table 3.1, R1–R11

R1 Accept a text-based PDF, politely reject anything else.

R2 Read text page by page, remember page numbers.

R3 Split the document into parts small enough for the AI model.

R4 Pull out facts from each part using the local AI model.

R5 Combine facts from all parts, remove repeats.

R6 Write a summary under nine headings, with page numbers for every point.

R7 Check the quality of the summary automatically.

R8 Show progress while it works; allow cancelling.

R9 Show the summary beside the original PDF; jump to any cited page.

R10 Keep a history of past analyses.

R11 Check at start-up that the AI model is installed; explain how to fix it if not.

3.3 Quality Requirements (four, non-functional)

Privacy (no internet used at all, even fonts are local) · cross-platform (same app, Windows + macOS) · honest about errors (names which step failed and why) · easy to change (model and folders live in one settings file) · repeatable (every analysis saved in its own timestamped folder).

3.4 Hardware and Software Requirements – Table 3.2

Key numbers to remember

≥8 GB RAM (tested on Apple M4, 24 GB) · Windows 10/11 or macOS (tested macOS 26.5) · Ollama running `qwen3.5:4b` (≈3.4 GB download) · Python 3.11+ with pypdf, Pydantic, FastAPI · desktop app: Electron + React.

5 Chapter 4: Detailed Design (report p. 5–7)

Big picture

Two systems exist in this report: the *full planned* system (Figure 4.1, three AI agents, a workflow engine, a search index) and the *Phase 1* system actually built (Figure 4.2, one agent, a simple 7-step pipeline, no search). Table 4.1 is the honest diff between the two, with *why* each simplification was made – this table is a very likely viva question.

4.1 The Complete Planned System – Figure 4.1

Desktop app (Electron) → Local server (FastAPI) → Workflow engine (Python), which runs three agents: **Requirements agent** (lists what the bidder must do), **Summary agent** (writes the executive summary – this is the only one built so far), **Starter Pack agent** (drafts a first outline of the proposal from past proposals). Everything is stored on the laptop: analysis records, a search index of documents, extracted text/results, an activity log.

4.2 What We Built in Phase 1 – Figure 4.2

Only the Summary agent. Flow: User → Desktop app → Local server (a queue, one job at a time, because the laptop can only run one AI model at once) → Analysis job (the 7 steps of Figure 4.3) → talks to the local AI model via Ollama → saves everything into one Results folder per analysis, which the server reads back to show progress/results.

4.3 Plan versus What Was Built – Table 4.1 (learn the "Why" column)

Part	Planned	Built	Why
Desktop app	Tauri	Electron	Needed a built-in PDF viewer for the side-by-side view; one language for the whole app
AI helpers	Three agents	Summary agent only	Get one right and measured first
Workflow engine	LangGraph [19]	A simple fixed sequence of 7 steps	One agent doesn't need a complex engine
Document search	Search index	Not needed yet	A summary reads the whole document anyway
Storage	Database	One folder per analysis	Easy to inspect; nothing to set up
Input files	PDF and Word	PDF only	Test RFPs were PDFs
User feedback	Accept/correct each point	View and check only	Planned for Phase 2

4.4 The Document Processing Pipeline – Figure 4.3 (the seven steps, memorize the order)

1. **Extract text** – read the PDF page by page.
2. **Split into parts** – keep sections together where possible.
3. **Analyze each part** ← uses the AI model (two focused questions per part).
4. **Combine results** – merge facts, remove repeats.
5. **Write the summary** ← uses the AI model (writes the nine headings).

6. **Check quality** – nine automatic checks.
7. **Show the report** – in the desktop app.

Only steps 3 and 5 touch the AI model; the rest is ordinary, deterministic program logic (fast, always gives the same answer for the same input).

4.5 Handling Long Documents – Figure 4.4

A small model can't read an 84-page RFP at once, so Klaro works like a team splitting up a long report: cut into ≈1,800-word parts (≈4 pages each), read separately, facts noted with page numbers, then all combined into one list that the Summary agent turns into the final summary. **Neighbouring parts overlap by 200 words** so a sentence cut right at a part boundary is not lost.

4.6 What the Summary Contains – Table 4.2 (the nine headings, memorize in order)

1. Overview – what is being bought and why, 2–3 sentences.
2. Issuing organization – who published the RFP.
3. Scope – the work and deliverables required.
4. Key requirements – documents/forms the bidder must submit.
5. Eligibility – who may bid: certificates, turnover, experience.
6. Important dates – deadlines, bid validity, review dates.
7. Commercial requirements – pricing, billing, payment terms, taxes, deposits.
8. Project duration – contract length and extensions.
9. Important conditions – penalties, confidentiality, ownership of work, exit/termination rules.

4.7 Keeping the Document Safe

The app only ever connects to programs on the same laptop (its own local server + the AI model runner). The renderer (screen part) is locked down and can only ask for three things: the server address, a file-picker dialog, and opening a link in the browser. The embedded PDF viewer can only show the current analysis's own RFP – nothing else. The summary panel can only ask to jump to a page number that actually exists in that document.

Key numbers to remember

7 pipeline steps (2 use the AI) · ≈1,800-word parts, 200-word overlap · 9 summary headings · 3 planned agents vs. 1 built.

6 Chapter 5: Implementation (report p. 8–10)

5.1 Tools Used – Table 5.1

Ollama [13] Runs the AI model on the laptop.

Qwen3.5 4B [14]
The AI model itself – reads the RFP parts, writes the summary.

Python + pypdf [15]
Reads text out of the PDF, page by page.

Pydantic [16]
Checks the AI's answers have the expected structure (schema validation).

FastAPI [17]
The local server between the desktop app and the analysis logic.

Electron + React [18]
The desktop app itself.

5.2 Reading and Splitting the PDF

Text read from each page, stored with its page number. Contents pages near the start are skipped. Headings are only recognized when they *look like* real headings (e.g. "4. SCOPE OF WORK" or "SECTION-D: …"); ordinary all-caps words are ignored, since RFPs are full of them. Each part is then built from whole sections where possible.

5.3 Asking the AI Two Focused Questions

Early tests showed asking the model for *everything* in one question gave poor results (missed items, cut-off answers). So each part is read **twice**: pass 1 asks about overview/scope/ requirements/eligibility; pass 2 asks about dates/money/duration/legal conditions. Every answer must follow a fixed structured format with a page number per point. Klaro tolerantly recovers usable points from a slightly malformed answer instead of discarding it. If the model fails on **more than half** the parts, the whole analysis is marked failed rather than silently continuing with missing data.

5.4 Combining Facts and Finding the Issuer

Facts from all parts grouped by heading, repeats removed, page numbers merged. **Issuer detection**: Klaro looks for the organization name mentioned most often (favouring names ending in words like Bank, Authority, Ministry). **Bug found and fixed**: the first version assumed a fixed default organization whenever it found no match, so every RFP got wrongly credited to the issuer of the very first test document. The fixed version now answers "Not identified" when unsure.

5.5 Writing and Checking the Summary

Combined facts given to the model with instructions to write all nine headings, keep page numbers, and use *only* the provided facts. If a heading comes back empty despite facts existing for it, those facts are copied in directly as a safety net so nothing is silently lost (this "safety copy" is exactly what causes the "too-good" scores discussed in Section 6.4.2). Automatic checks then ask: does every heading exist? is every point backed by a fact? does every point have a page number? how many facts made it in? are any points repeated? how much shorter is the summary than the raw facts? is anything under the wrong heading? These checks compare *words*, not meaning – they confirm the summary agrees with the extracted facts, not that the extraction itself was correct.

5.6 The Desktop Application – Figures 5.1–5.3

On open, the app starts the local server in the background and waits until it responds; if the model is missing, a setup screen explains how to install it. **Figure 5.1** (user flow): Open Klaro → [AI ready? no → Setup screen; yes →] Choose an RFP → Watch progress (can cancel) → Read the summary → Click a page number → PDF opens at that page → Checked. **Figure 5.2**: home screen – drag/drop or choose a PDF, table of past analyses with pages, chunks, duration, status. **Figure 5.3**: progress screen – live step list with per-step timing and a percentage bar for the current step ("Chunk 2 of 14").

5.7 Testing

Fifteen automatic tests cover: report-page rendering, the order of pipeline steps (orchestration), handling of failed parts, and issuer detection. All fifteen pass. The AI steps were tested end-to-end on the real documents used in Chapter 6. App tested by hand on macOS; a teammate tests Windows.

Key numbers to remember

2 focused questions per part · fail-analysis threshold = more than half the parts failing · 15 automatic tests, all passing · 7 automatic quality checks on the summary.

7 Chapter 6: Experimental Results and Analysis (report p. 11–14)

Big picture

Two separate experiments happen in this chapter: (A) a **model benchmark** to pick which AI model Klaro should use (Section 6.1, Tables 6.1–6.3), and (B) **full end-to-end runs** of the finished app on real RFPs (Section 6.3, Table 6.4). Do not mix these two tables up in the viva – 6.1–6.3 are about *choosing the model*; 6.4 is about *testing the finished product*.

6.1 Choosing the AI Model

Three candidate small models compared: **Gemma 4 (e4b)**, **Qwen3.5 (4B)**, **Phi-4-mini**. Same 20 RFP parts given to each; asked to list requirements; compared against 47 hand-marked (ground-truth) requirements.

Table 6.1 – Accuracy (47 hand-marked requirements):

	Gemma 4 (e4b)	Qwen3.5 (4B)	Phi-4-mini
Requirements found (of 47)	33	23	7
Wrong/extra items listed	22	**8**	9
Precision	60%	**74%**	44%
Recall	**70%**	49%	15%
F1-score	**65%**	59%	22%
Mandatory vs. optional, correct	85%	**91%**	57%
Made-up requirements	0%	0%	6%

Table 6.2 – Size, speed, memory: Gemma = 8.0B params / 9.6 GB download / 6.7s per part typical; Qwen3.5 = 4.7B params / 3.4 GB / 8.1s per part; Phi-4-mini = 3.8B params / 2.5 GB / 1.3s per part (fastest). Peak memory: Gemma 4.4 GB, Qwen 6.0 GB, Phi 4.3 GB.

Table 6.3 – Following the answer format: all three models were usable 100% of the time after Klaro's tolerant reading, but Qwen followed the *exact* requested format 0% of the time (it gave a bare list 30% of the time) versus Gemma 45% exact / Phi 35% exact.

Why Qwen3.5 was chosen (this is the single most important paragraph in Chapter 6)

- Phi-4-mini ruled out immediately: fastest, but found only 7/47 requirements and was the *only* model that invented (hallucinated) a requirement.
- Between Gemma and Qwen: Gemma found more and was faster (higher F1, 65% vs. 59%), **but** it is a much bigger model – 8B params, 9.6 GB, almost 3× Qwen's 3.4 GB. Every user has to download and store this, so a smaller model matters for a laptop app.
- Qwen was the *more careful* model: 74% precision vs. Gemma's 60%, only 8 wrong items vs. Gemma's 22, and best at judging mandatory vs. optional (91%).
- **Core argument**: for a summary people rely on, a *wrong* point does more harm than a *missing* one – a missing point can be recovered by asking more focused questions (which is exactly why Klaro asks two question-sets per part), while a wrong point may go unnoticed.
- Qwen's weakness (not following exact format) is handled separately, by Klaro's tolerant answer reading – so it isn't a real cost.
- Because model choice is one config setting, Gemma can be swapped in any time for comparison; with improved instructions its score later rose to 67%, so it remains the main alternative.

6.2 First Experiments and the Pilot Run

Before the two-question design existed, the very first attempt asked for *all* requirements in one single question per part. Result on 5 parts: 2 empty answers, 1 cut off/unusable, single answers took 2–10 minutes. This directly caused the two-question redesign. The first full pilot run (by a teammate, old single-question-per-part-analysis but two-question design) on the 84-page RFP: 51 points, every one backed by facts with a page number, 0 made-up info, 76% of facts included, summary 36% shorter than the facts. **9 points were filed under the wrong heading** (e.g. a CMMI certificate under "conditions" instead of "eligibility") – this is the concrete evidence behind the Chapter 2 discussion of Knisely & Pavliscsak.

6.3 Full Runs through the Desktop App – Table 6.4

Finished app run on **two RFPs across three runs**: 84-page government RFP for software development services (R1); 54-page bank RFP for a technical service provider, run **twice** (R2, R3 – to test run-to-run consistency).

	R1	**R2**	**R3**
Pages	84	54	54
Parts analyzed	17	14	14
Total time	22m 51s	25m 19s	22m 16s
Points in summary	170	211	216
Backed by facts / page numbers	100% / 100%	100% / 100%	100% / 100%
Facts included in summary	99%	97%	96%
Shorter than the facts by	8%	8%	9%
Made-up information	none	none	none
Parts where AI failed	0	0	0

6.4 What the Results Show

6.4.1 Time Taken

≈95% of total time is step 3 (AI reads each part twice, ≈1.5 min/part); writing the summary adds ≈1 more minute. So a 50–85 page RFP = 22–25 minutes total.

6.4.2 Why Some Scores Look Too Good

The most important finding in the whole report. Comparing files showed every point under the seven list headings was an *exact copy* of a collected fact – for long RFPs the model only wrote the Overview heading itself, and the "safety copy" fallback from Section 5.5 filled in the rest. So for a long RFP, the "summary" right now is really a cleaned-up list of facts, not a genuine summary – and the "100% backed by facts" score is high *partly because of that copying*, not purely because the model is accurate. Likely cause: the fact list for a long RFP is too long for the amount of text the model reads at once. Not yet confirmed – it's the first fix planned for Phase 2.

6.4.3 Same Document, Different Answers

R2 and R3 = identical PDF, but the AI found 236 facts the first time and 244 the second, with some heading counts differing by up to 10. Lesson: AI output is not perfectly deterministic, so improvements must be judged over *several* runs, not one.

6.4.4 Issuer Detection

Concrete before/after: with the old code, the bank RFP (R2/R3) would have been wrongly credited to the government agency from R1 (the first test document). After the fix, it correctly returned "India Post Payments Bank Limited,"

with the ten pages where that name appears.

6.4.5 Checking the Summary

Every point's page number is clickable and opens the source PDF at that page (Figure 6.2). The Quality tab (Figure 6.3) shows Grounding, Traceability, Coverage, Duplication scores, with an explicit note that these compare *words*, not meaning. No internet connection was made during any run.

Key numbers to remember

Model chosen: **Qwen3.5 (4B)**, 74% precision, 49% recall, 59% F1, 3.4 GB. · 2 RFPs / 3 runs · 22–25 min per run · 100% points backed & page-numbered, every run · 0 made-up info, every run · biggest known weakness: long-RFP summaries are really just copied facts (Section 6.4.2).

8 Chapter 7: Conclusion and Future Scope (report p. 15–16)

7.1 Conclusion

One-paragraph project summary: Phase 1 works end-to-end (54–84 page tender in, 22–25 minutes later a nine-heading, page-referenced summary out, nothing sent over the internet). Model chosen by testing three on the same task; Qwen3.5 won on fewest wrong claims and ≈1/3 the size of its closest competitor. Same pipeline runs from the CLI (for testing) and inside the desktop app (for analysts); failures are surfaced clearly. Testing exposed the long-RFP copying weakness, which sets the agenda for Phase 2.

7.2 Limitations (five – be ready to state all five)

1. Only three RFPs tested so far – can't yet generalize to tenders in general.
2. No human-written reference summaries exist yet to compare against.
3. Long-RFP summaries aren't genuinely shortened yet (Section 6.4.2).
4. AI answers vary run to run, and 22–25 minutes is slow.
5. Only English, text-based PDFs work; tables lose their layout.

(The report also notes: no installer yet – the app runs from the project folder.)

7.3 Future Scope

7.3.1 Proposal Assistant (Figure 7.1)

Two planned modes. **Agentic workflow**: three AI helpers share one reading of the RFP – Requirements agent (checklist), Executive Summary agent (existing), Starter Pack agent (drafts a first proposal outline with risks/questions). **File search workflow**: past proposals/case studies are indexed locally; the most similar ones to a new RFP are found and handed to the Starter Pack agent.

7.3.2 Real-Time Progress Updates

Show partial results *as* each step produces them (facts appearing live, summary streaming as it's written) instead of only a step name.

7.3.3 Better Summaries and Better Testing

(a) Fix the long-document copying problem: read more at once, write one heading at a time, record when the safety copy is used. (b) Build a 30–40 RFP reference set: a larger LLM drafts a first summary, a human corrects it, the corrected version becomes the "golden" summary used to test every future change. (c) Fine-tune Qwen3.5 with LoRA on those golden summaries (cheap fine-tuning, not full retraining) and compare against the current model – this directly targets the 6.4.2 copying problem. (d) Repeat the model benchmark, keeping Gemma 4 as the main alternative.

7.3.4 App Improvements

One-click installers (Win + macOS) with guided model setup; support for Word files, scanned PDFs, tables; **multilingual support starting with German** (read German RFPs, reply in German or English – first added language, mid-term goal); let users accept/edit/flag each point and use that feedback to measure quality.

Key numbers to remember

5 limitations · Proposal Assistant = 2 workflows (agentic + file search), 3 agents total · Phase-2 plan = fix copying + 30–40 RFP golden set + LoRA fine-tune + repeat benchmark · German = first planned language.

9 References – what every paper actually says (report p. 17)

Big picture

References [1]–[12] are genuine research papers; [13]–[19] are tool/library documentation pages (Ollama, Qwen3.5, pypdf, Pydantic, FastAPI, Electron, LangGraph), not papers – listed here for completeness since they're in the References list too.

[1] Beason, Hinton, Salamah & Salsman – "Automated analysis of RFPs using NLP for the technology domain" (SMU Data Science Review, 2021)

Processed a large set of real US Department of Defense and Homeland Security RFPs with NLP, looking for technology trends across the whole collection (not summarizing any one RFP). Their two big findings, both directly reused by Klaro: RFPs follow no consistent layout and contain a lot of noise (contents pages, boilerplate, legal text) that must be stripped out first; and readers get more value from exact, structured facts than from a free-flowing prose summary.

[2] Trappey et al. – "Intelligent RFQ summarization using NLP, text mining, and machine learning techniques" (Journal of Global Information Management, 2022)

Summarized 1,300+ power-transformer procurement tenders (RFQs) using a two-stage pipeline: TF-IDF to find important keywords, then TextRank to rank and select sentences by topic. Keeping only the top 35% of ranked sentences preserved about 80% of the important requirements while cutting reading time to roughly 29% of the original. Klaro borrows the "summarize per topic, not all at once" idea and the goal of a genuine length reduction (a goal Klaro has not yet met for long RFPs – see Section 6.4.2).

[3] Mihalcea & Tarau – "TextRank: Bringing order into text" (EMNLP, 2004)

The foundational graph-based ranking algorithm cited by [2]. Builds a graph where sentences (or words) are nodes, connected by edges weighted by similarity or co-occurrence; then runs a PageRank-style random-walk algorithm over that graph to score how "central"/important each node is. The highest-scoring sentences (or words) become the extractive summary (or keyword list). It needs no training data and no deep model – purely statistical/graph-based, which is why it is cheap enough to run on 1,300+ documents in [2].

[4] Knisely & Pavliscsak – "Research proposal content extraction using NLP and semi-supervised clustering" (Scientometrics, 2023)

Sorted a large set of research grant proposals into themes using semi-supervised clustering: a small number of proposals were labelled by human experts, and those labels guided an algorithm in sorting the rest automatically. This worked substantially better than clustering with no human guidance at all. Klaro connects this to its own finding that the AI sometimes misfiles a fact under the wrong heading (Section 6.2) – both point toward a little targeted human correction going a long way, which is exactly why Phase 2 starts with a small human-checked reference set rather than pure automation.

[5] Vaswani et al. – "Attention is all you need" (NeurIPS, 2017)

The paper that introduced the **Transformer** architecture – the design behind essentially every modern AI language model, including Gemma, Qwen and Phi, the three models Klaro benchmarks. Its core idea is **self-attention**: instead of reading text strictly left-to-right (like older recurrent models), every word can directly look at every other word in the input to decide what matters,

enabling parallel training and much better handling of relationships between distant words. The catch, relevant to Klaro's whole chunking design: the computational cost of self-attention grows *quadratically* with input length, so a standard Transformer can only look at a limited amount of text at once – exactly the constraint cited in report Section 2.4 that forces Klaro to split long RFPs into parts.

[6] Beltagy, Peters & Cohan – "Longformer: The long-document transformer" (arXiv, 2020)

Redesigns self-attention so its cost grows *linearly* (not quadratically) with input length, by combining local attention (each word attends mainly to nearby words) with a small amount of global attention (a few special tokens that attend to everything). This lets a Transformer read thousands of words in a single pass. Report's honest caveat: this still needs more powerful hardware than an ordinary laptop, which is why Klaro chunks instead of adopting a Longformer-style model.

[7] Douzon, Duffner, Garcia & Espinas – "Long-range transformer architectures for document understanding" (ICDAR 2023 Workshops)

Extends the long-context idea specifically for business documents, by also feeding the model the document's page layout (positions of headings, tables, columns) so that, for example, a value inside a table stays correctly linked to its column/row heading rather than being read as flat text. More capable for structured documents, but heavier to run – report's stated reason it's "too heavy for a laptop."

[8] Ji et al. – "Survey of hallucination in natural language generation" (ACM Computing Surveys, 2023)

A broad survey paper cataloguing the different ways language-generation models produce **hallucinations** – statements not supported by (or contradicting) the source input – across many NLP tasks, along with the known causes (e.g. training-data noise, the model's own generation process) and mitigation strategies proposed in the literature. Cited as the general evidence behind Klaro's Section 2.5 statement that "language models sometimes state things that are not in the text they were given."

[9] Maynez, Narayan, Bohnet & McDonald – "On faithfulness and factuality in abstractive summarization" (ACL, 2020)

An empirical study specifically on *summarization* models (directly relevant to Klaro): shows that abstractive summarizers frequently generate content that is not supported by, or directly contradicts, the source document, and proposes ways to actually measure this "faithfulness" rather than just measuring fluency. This is the direct research justification for why Klaro's automatic quality checks (Section 5.5, Figure 6.3) exist at all – checking that every summary point is "grounded" in an extracted fact.

[10] Liu et al. – "Lost in the middle: How language models use long contexts" (TACL, 2024)

Shows that language models are noticeably better at using information placed at the very *beginning* or *end* of a long input than information buried in the *middle* – a U-shaped performance curve as position in the context varies. Cited in Klaro Section 2.5 as a second reason (besides hallucination) that simply throwing a whole long RFP at a model in one go is risky – reinforcing the case for Klaro's chunk-by-chunk reading instead of one giant prompt.

[11] Lewis et al. – "Retrieval-augmented generation for knowledge-intensive NLP tasks" (NeurIPS, 2020)

Introduced **RAG**: pair a *retriever* (which searches a large corpus and fetches the most relevant passages for a given query) with a *generator* (a language model that only has to read those retrieved passages, not the whole corpus). This directly reduces hallucination, since the model is only ever shown text that is actually relevant, and its answers can be traced back to specific retrieved passages. Klaro's own design (cite a page for every point, then check each point against the collected facts) is described in the report as its own answer to the same underlying problem RAG solves, without needing a full retrieval index for a single-document task.

[12] Hu et al. – "LoRA: Low-rank adaptation of large language models" (ICLR, 2022)

Introduces **LoRA (Low-Rank Adaptation)**: instead of updating all of a large pretrained model's weights during fine-tuning (expensive, and requires storing a full new copy of the model per task), LoRA freezes the original weights entirely and injects small, trainable *low-rank* matrices into specific layers (typically the attention projections). Only these small matrices are trained, which drastically cuts the number of trainable parameters and the compute/memory needed to fine-tune, while keeping inference speed almost unchanged. This is exactly the technique named in Klaro's Phase-2 plan (Section 7.3.3) for fine-tuning Qwen3.5 on the 30–40-RFP golden-summary reference set.

[13]–[19] Tool documentation (not research papers)

[13] Ollama
Official docs for the tool that runs AI models locally on the laptop.

[14] qwen3.5 model library entry
Ollama's page for the specific model Klaro uses.

[15] pypdf documentation
The Python library used to read text out of PDFs.

[16] Pydantic documentation
The Python library used to validate the AI's structured answers.

[17] FastAPI documentation
The Python web-framework used for Klaro's local server.

[18] Electron security documentation
Cited specifically for the security practices (context isolation, no direct Node access, etc.) behind Section 4.7.

[19] LangGraph documentation
The workflow-engine library that was *planned* for the full multi-agent system (Figure 4.1) but not used in Phase 1, per Table 4.1.

10 Cheat Sheet – every number in one place

Fact	Number
Test RFP lengths	54 and 84 pages
Summary headings	9 (fixed, always in the same order)
Objectives	4
Limitations listed	5
Papers reviewed in Background Study	5 close papers, 19 references total
Functional requirements	R1–R11 (11 total)
Pipeline steps	7 (steps 3 and 5 use the AI model)
Chunk size / overlap	≈1,800 words per part / 200-word overlap
Model benchmark test set	20 RFP parts, 47 hand-marked requirements
Gemma 4 (e4b)	33/47 found, 60% precision, 70% recall, 65% F1, 8.0B params, 9.6 GB
Qwen3.5 (4B) – **chosen**	23/47 found, 74% precision, 49% recall, 59% F1, 4.7B params, 3.4 GB
Phi-4-mini – ruled out	7/47 found, 44% precision, 15% recall, 22% F1, only model to hallucinate (6%)
Gemma's later improved score	67% (with better instructions) – remains main alternative
Pilot run (84-page, old single-question)	51 points, 76% facts included, 36% shorter, 9 misfiled
Full run R1 (84 pg, govt)	22m 51s, 170 points, 99% facts included, 8% shorter
Full run R2 (54 pg, bank, 1st)	25m 19s, 211 points, 97% facts included, 8% shorter, 236 facts found
Full run R3 (54 pg, bank, 2nd)	22m 16s, 216 points, 96% facts included, 9% shorter, 244 facts found
Made-up info across all 3 full runs	0 (none)
Points backed by facts / page numbers, all 3 runs	100% / 100%
Time breakdown	≈95% of time = AI reading (step 3), ≈1.5 min/part
Automatic tests	15 (all pass)
Automatic quality checks per summary	7 questions (headings exist? backed? page numbers? coverage? duplicates? shorter-by-how-much? right heading?)
Quality-tab scores (Figure 6.3 example)	Grounding 100%, Traceability 100%, Coverage 97%, Duplication 3.8%
Phase-2 fine-tuning dataset plan	30–40 RFPs, human-corrected golden summaries, LoRA fine-tuning
Minimum RAM	8 GB (tested on 24 GB Apple M4)